SOUFER Tools — Integração da Interface com a API (Entrega 27/10/2026)

Disciplina: Desenvolvimento de Interfaces de Usuário para Web **Projeto Integrador:** SOUFER Tools — controle web de retirada e devolução de ferramentas da manutenção da Soufer **Atividade:** Integração da Interface com a API — Prof. Marcelo (27/10/2026)

Nome	RA
Guilherme Portilho da Rosa Santi	25000151
Henrique de Oliveira Molinari	25001176
João Augusto de Freitas	25000019
Kauan Leander Leandrini	25000795

---

1. Contexto

Esta entrega atende à atividade do projeto de extensão de 27/10/2026: integrar o front-end aos serviços da API para que as principais funcionalidades usem **dados reais**, demonstrando **formulários com validação**, **consumo dos endpoints**, **tratamento das respostas** e **atualização da interface** a partir das operações do usuário.

Na entrega anterior (P1, 02/10/2026) as telas usavam dados simulados e a tela de login tinha um botão "Entrar sem a API (demonstração)". Esse botão **foi removido**: hoje toda tela consome a API RESTful do projeto (prefixo `/v1`), autenticada por JWT, e não existe nenhum dado simulado no código do `/web`.

O SOUFER Tools acompanha qual ferramenta está disponível, em uso ou indisponível, quem retirou o quê, e abre uma ocorrência automaticamente quando a ferramenta volta com avaria ou é dada como perdida. O front fica em `/web` e a API em `/api`, no mesmo monorepo.

1.1 Como as evidências foram produzidas

Os prints e as respostas da API foram gerados em **27/10/2026** no ambiente local da equipe: front (Vite, `localhost:5173`) falando com a API (`localhost:3000/v1`) e o banco PostgreSQL `soufer_dev`, todos na mesma versão do código da branch `development`. Os fluxos foram executados pela própria interface, de ponta a ponta, num navegador automatizado (Chrome), sem nenhum dado fabricado: o que aparece nos prints é o que a API devolveu.

Foram criados três registros de teste, identificáveis pelo prefixo **DEMO ENTREGA**: dois usuários (matrícula 9801, perfil manutenção, e 9802, perfil admin), um colaborador (9803) e uma ferramenta (código 000094).

O front publicado continua em https://soufer-tools.servidorcaseiro.online/.

2. Tecnologias

Camada	Tecnologia
Build/dev server	Vite 8
Framework	React 19 + TypeScript
Roteamento	React Router 7
Dado remoto/cache	TanStack Query
Cliente HTTP	Axios (instância única em `web/src/lib/api.ts`)
Formulários e validação	React Hook Form + Zod
Estilização / componentes	Tailwind CSS v4 + shadcn/ui
Notificações	sonner (toasts)
Gráficos / código de barras	Recharts / react-barcode (Code128)
API	Node 20 + Express + `pg` + Zod + JWT (`jsonwebtoken` + `bcryptjs`)
Banco	PostgreSQL

3. Como a integração está organizada

- **Um cliente HTTP só.** Toda chamada passa por `web/src/lib/api.ts` (Axios com `baseURL` vinda de `VITE_API_URL`). Um interceptor de requisição anexa o token JWT e um interceptor de resposta trata o `401` (encerra a sessão e volta ao login).
- **Um hook por recurso.** Os hooks em `web/src/hooks/` envolvem o TanStack Query: `useQuery` para leitura (com cache) e `useMutation` para escrita. As páginas não chamam `api` diretamente, com poucas exceções (convite de acesso, status da API, importação de CSV e impressão de pendências).
- **Validação antes de enviar.** Cada formulário tem um schema Zod ligado ao React Hook Form (`zodResolver`). O servidor valida de novo com Zod: o front evita a ida e volta, mas quem garante a regra é a API (e, em alguns casos, o banco).
- **Erro padronizado.** A API responde sempre no envelope `{ "error": { "code", "message", "details" } }`. O front lê esse envelope em `web/src/lib/avisar-erro.ts` e mostra a mensagem em um toast vermelho.
- **Interface sempre atualizada.** Depois de cada operação de escrita, o hook invalida as chaves de cache afetadas (`invalidateQueries`) e o TanStack Query refaz as consultas: listas, contadores do menu, dashboard e notificações mudam sem recarregar a página.

4. Mapa tela → endpoint → hook

Tela (rota)	Endpoints consumidos	Hook / origem
Login (`/login`)	`POST /auth/login`	`AuthProvider` (`lib/auth.tsx`)
Convite de acesso (`/c/:token`)	`GET /auth/convites/:token` , `POST /auth/convites/:token/senha`	chamada direta na página
Consulta / quiosque (`/consulta`)	`POST /consulta/sessao` , `GET /consulta/ferramentas`	`useIniciarSessaoConsulta` , `useConsultaFerramentas`
Dashboard (`/`)	`GET /dashboard` , `GET /dashboard/:lista`	`useDashboard` , `useListaDashboard`
Ferramentas (`/ferramentas`)	`GET /ferramentas` , `GET /categorias`	`useFerramentas` , `useCategorias`
Nova ferramenta (`/ferramentas/nova`)	`POST /ferramentas` , `GET /categorias` , `GET /setores`	`useCriarFerramenta` , `useCategorias` , `useSetores`
Detalhe da ferramenta (`/ferramentas/:id`)	`GET /ferramentas/:id` , `GET /ferramentas/:id/historico` , `PATCH /ferramentas/:id/etiqueta-impressa`	`useFerramenta` , `useHistoricoFerramenta` , `useMarcarEtiquetaImpressa`
Registrar retirada (`/retiradas/nova`)	`GET /ferramentas/por-codigo/:codigo` , `GET /ferramentas` , `GET /colaboradores/identificar` , `POST /colaboradores` (cadastro rápido), `POST /emprestimos`	`useFerramentaPorTermo` , `useColaboradorPorTermo` , `useCadastrarColaboradorRapido` , `useRetirarFerramenta`
Registrar devolução (`/devolucoes`)	`GET /emprestimos` (abertos), `PATCH /emprestimos/:id/devolucao` , `PUT /ocorrencias/:id/foto`	`useEmprestimosAbertos` , `useEmprestimoAberto` , `useDevolverEmprestimo`
Indisponíveis (`/indisponiveis`)	`GET /ferramentas` , `GET /ferramentas/:id/historico` , `PATCH /ocorrencias/:id/avancar` , `PATCH /ferramentas/:id/disponibilizar` , `DELETE /ferramentas/:id`	`useFerramentas` , `useAvancarTratativa` , `useDisponibilizarFerramenta` , `useBaixarFerramenta`
Calendário (`/calendario`)	`GET /emprestimos/calendario`	`useCalendario`
Histórico (`/emprestimos`)	`GET /emprestimos`	`useEmprestimos`
Análises (`/analises` , admin)	`GET /kpis/:rota`	`useKpi`
Sino de notificações (todas as telas)	`GET /notificacoes` , `PATCH /notificacoes/:id/lida` , `PATCH /notificacoes/lida`	`useNotificacoes` , `useMarcarLida` , `useMarcarTodasLidas`

Tela (rota)	Endpoints consumidos	Hook / origem
Cadastros — colaboradores, ferramentas, categorias, setores (`/cadastros/*` , admin)	`GET/POST/PATCH/DELETE /:recurso` , `PUT/DELETE /ferramentas/:id/foto` , `POST /colaboradores/:id/convite` , `POST /:recurso` (importação de CSV)	`useListaCadastro` , `useSalvarCadastro` , `useInativarCadastro` , `useEnviarFoto` , `useRemoverFoto`
Status da API (`/status`)	`GET /health`	chamada direta na página

5. Formulários com validação

A validação roda em três camadas: no navegador (Zod + React Hook Form, mensagem ao lado do campo), na API (Zod em todas as rotas de escrita, resposta `400 VALIDATION_ERROR`) e no banco (índices únicos e `CHECK` , resposta `409`).

Formulário	Regras no front (schema Zod)	Evidência
Login	Matrícula de 4 dígitos; senha (PIN) de 6 dígitos	Figuras 1, 2 e 3
Nova ferramenta	Nome com 2 a 150 caracteres; categoria obrigatória	Figuras 4, 5 e 6
Cadastro de colaborador	Matrícula de 1 a 4 dígitos; nome obrigatório (até 150); setor obrigatório	Figuras 7 a 11
Retirada	Ferramenta e colaborador identificados; setor e previsão de devolução obrigatórios; atividade opcional	Figuras 13 a 16
Devolução	Condição OK, avaria ou perda; com avaria ou perda a descrição é obrigatória	Figuras 19 a 22

Fluxo 1 — Login (validação, erro de credencial e sucesso)

Sem preencher nada, o botão **Entrar** mostra "A matrícula tem 4 dígitos" e marca os campos (Figura 1). Com matrícula válida e senha errada, o formulário é aceito pelo front, a API responde **401** e a tela mostra "Matrícula ou senha inválidas." e limpa o PIN (Figura 2). Com a senha correta, a API devolve o JWT, a sessão é guardada e o usuário cai no dashboard (Figura 3).

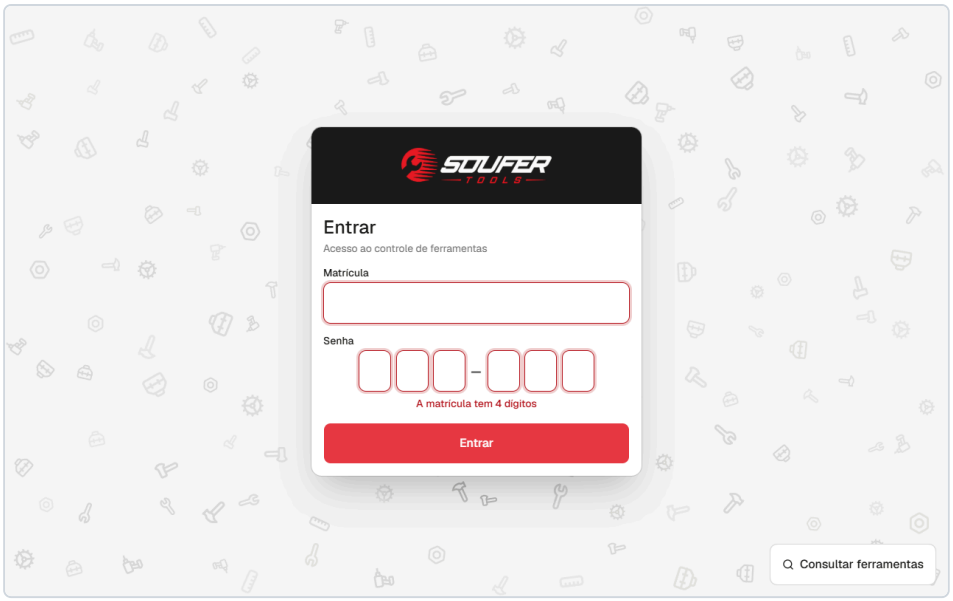


Figura 1 — Login: validação de campos vazios

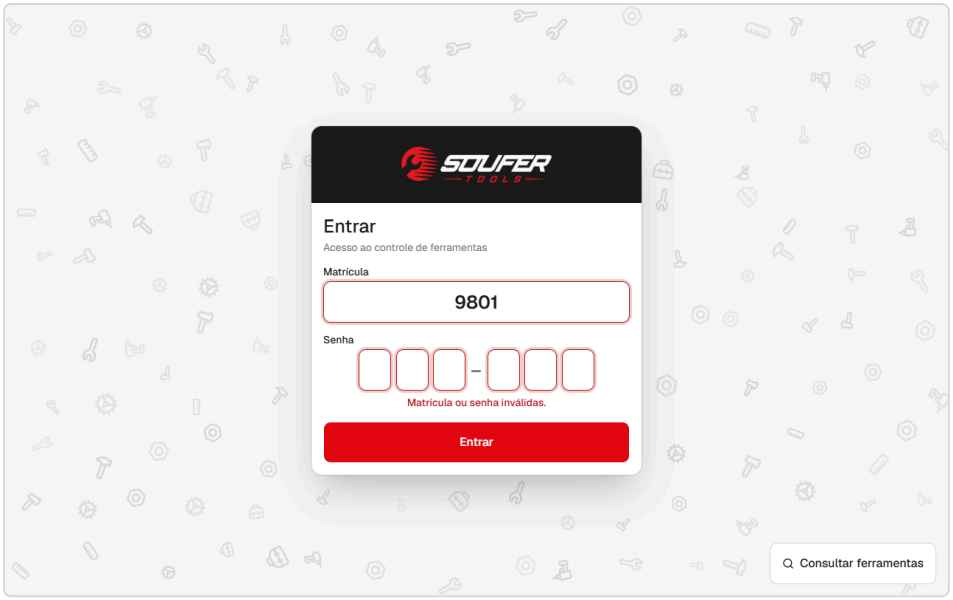


Figura 2 — Login: API respondeu 401 e a tela mostrou o erro

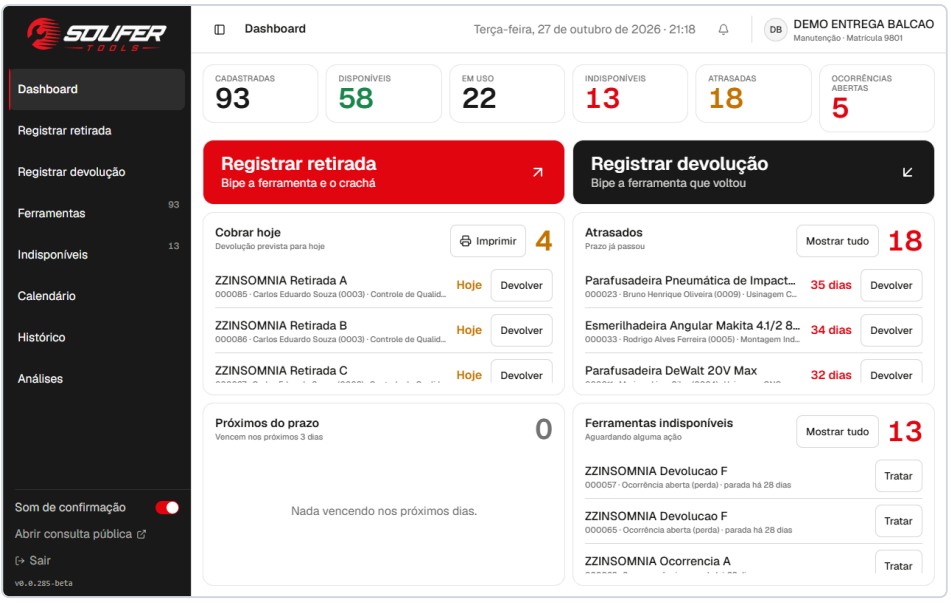


Figura 3 — Login com sucesso: dashboard com dados reais

Fluxo 2 — Cadastro de ferramenta (POST /ferramentas)

O envio vazio é barrado no front: "Informe o nome da ferramenta" e a categoria fica destacada (Figura 4). Com o formulário preenchido (Figura 5), a API responde **201**, o sistema mostra o toast "cadastrada como 000094" e abre o diálogo para imprimir a etiqueta Code128 (Figura 6).

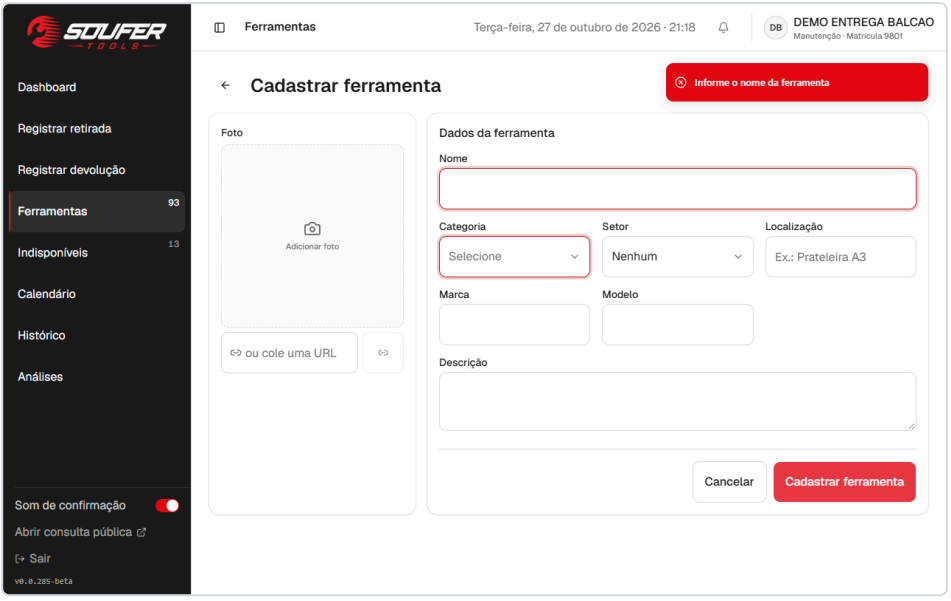


Figura 4 — Nova ferramenta: validação

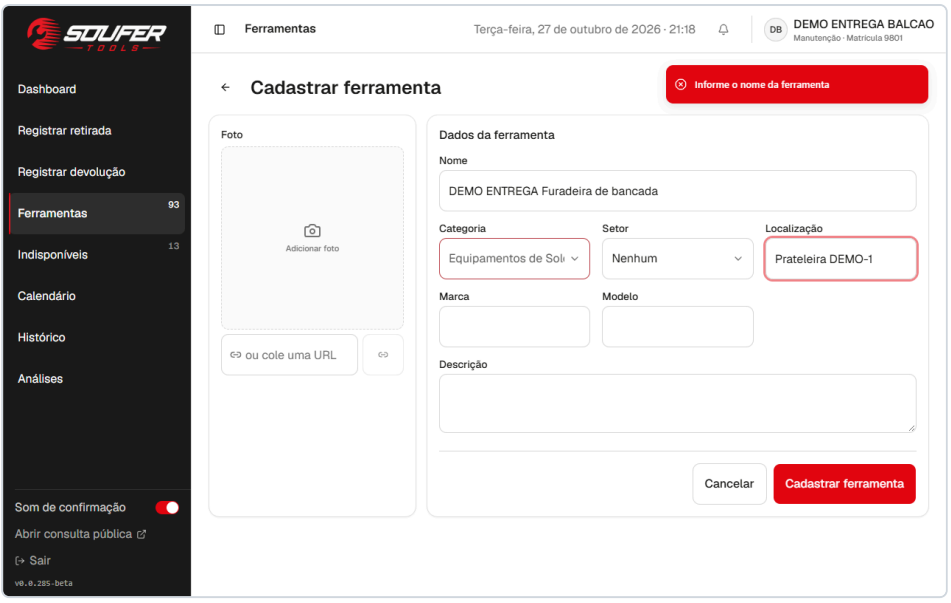


Figura 5 — Nova ferramenta: formulário preenchido

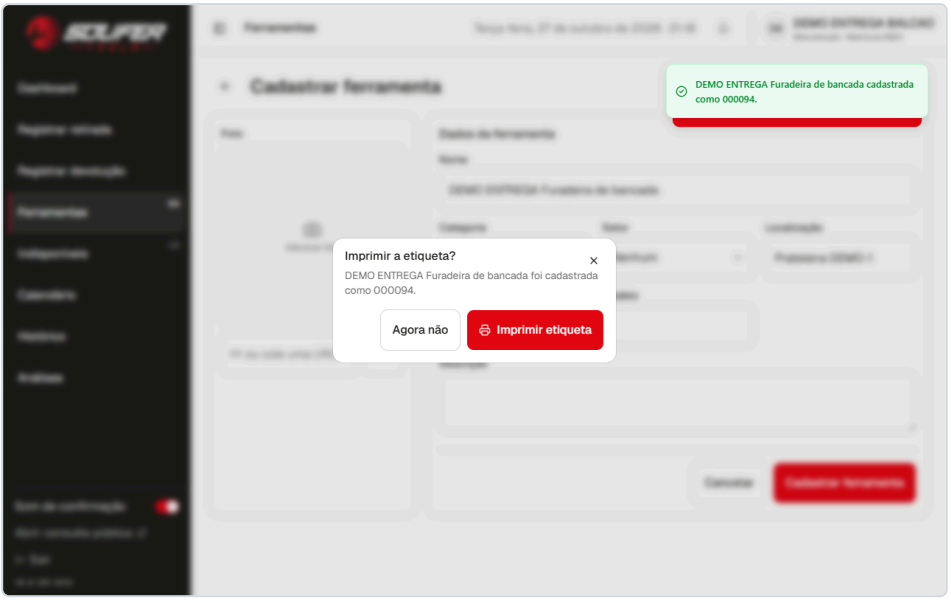


Figura 6 — Nova ferramenta: criada pela API (201) e etiqueta oferecida

Fluxo 3 — Cadastro de colaborador pelo admin (400 e 409 tratados)

O admin abre **Cadastros** → **Colaboradores** (Figura 7). Enviar o diálogo vazio mostra os três erros de campo (Figura 8). Preenchido (Figura 9), o cadastro volta **201** e o toast "Colaborador cadastrado." (Figura 10). Repetir a mesma matrícula faz o **banco** recusar a duplicidade (índice único); a API traduz para **409** `DUPLICATE_ENTRY` e o front mostra o aviso vermelho sem fechar o diálogo (Figura 11), de modo que o usuário corrige sem perder o que digitou. A resposta crua da API está em `evidencias/01-api-respostas-de-erro.txt`.

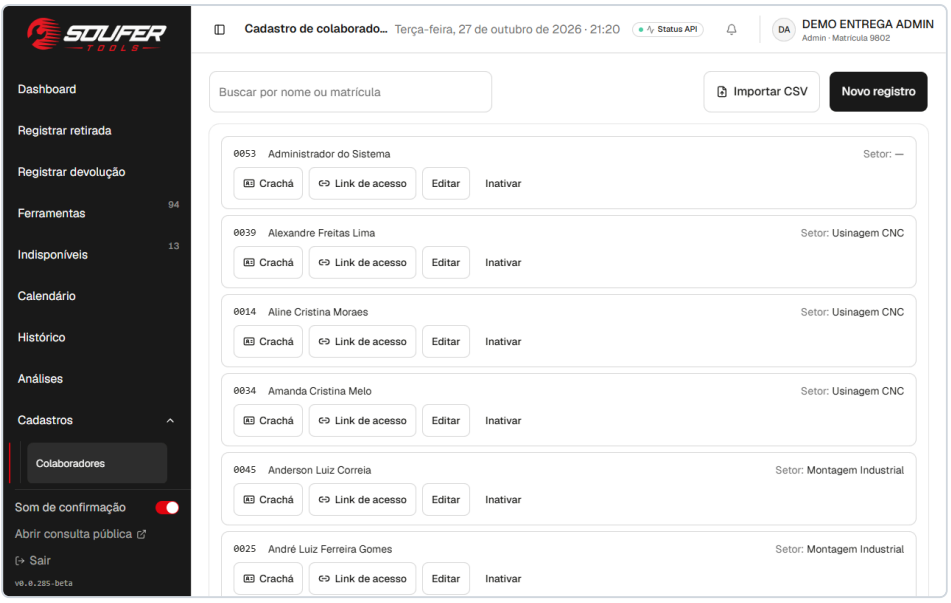


Figura 7 — Cadastros: lista de colaboradores vinda da API (perfil admin)

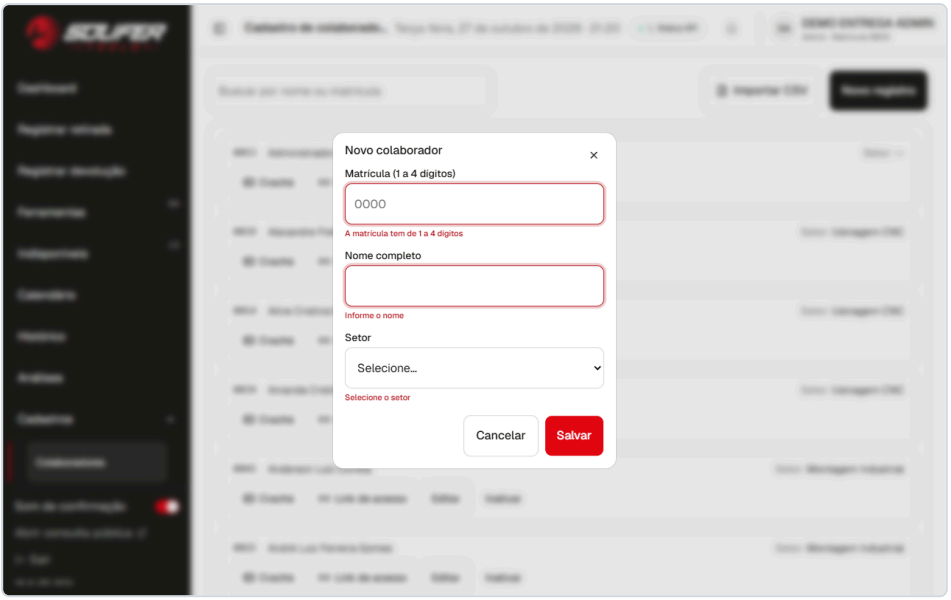


Figura 8 — Colaborador: validação dos três campos

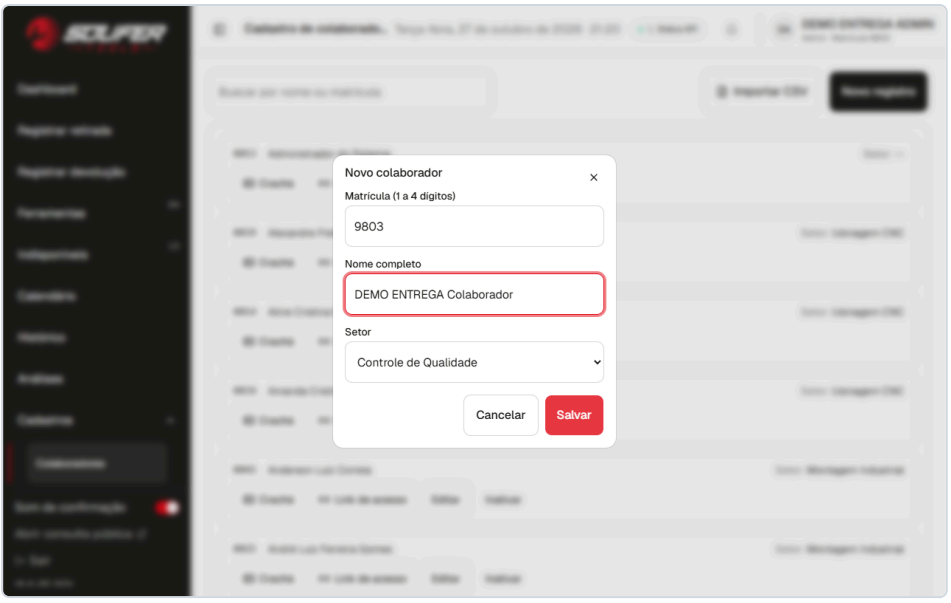


Figura 9 — Colaborador: formulário preenchido

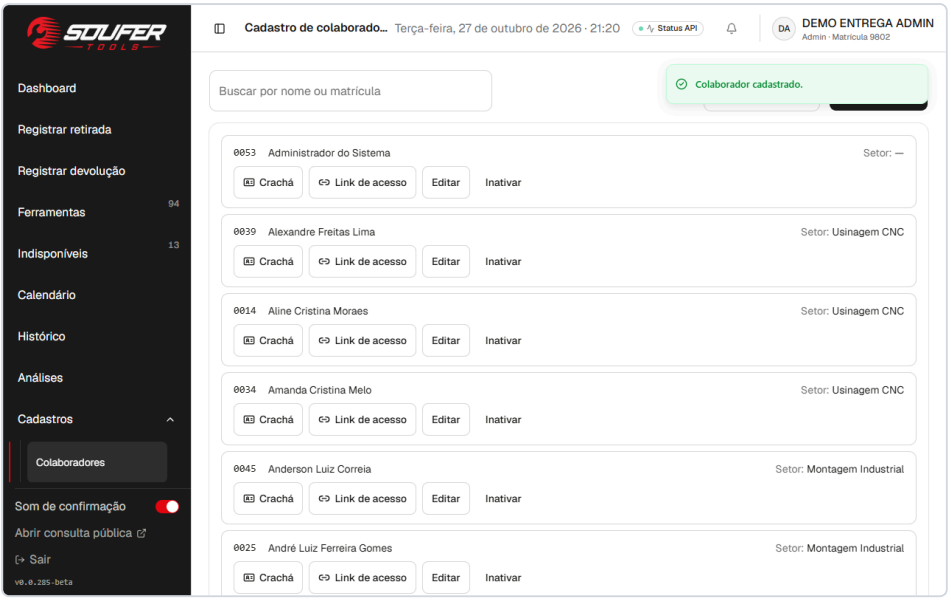


Figura 10 — Colaborador: criado (201) e lista atualizada

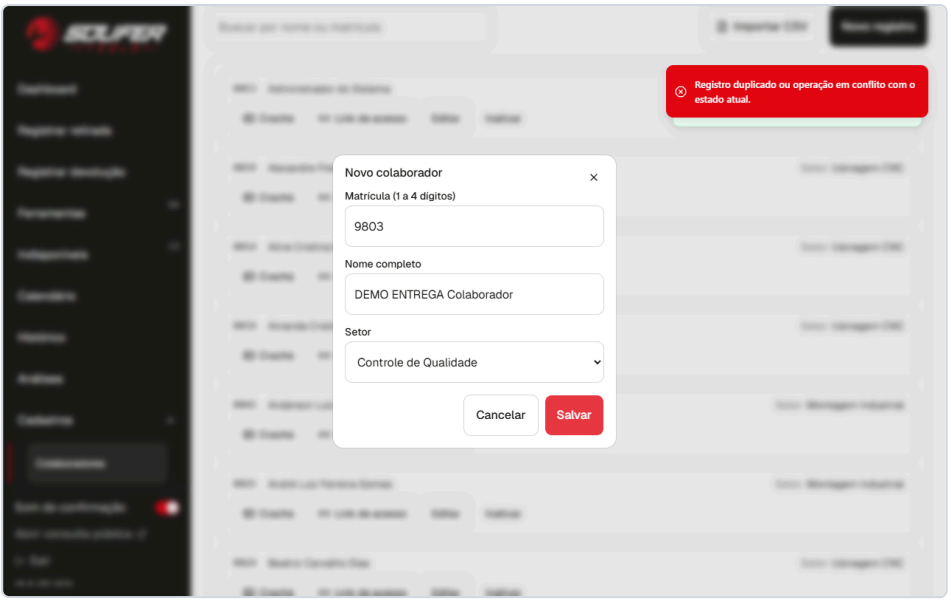


Figura 11 — Colaborador: matrícula duplicada, a API respondeu 409 e a tela avisou

6. Operações com atualização da interface

Fluxo 4 — Retirada de ferramenta (POST /emprestimos)

Antes da retirada o dashboard marcava **59 disponíveis, 22 em uso e 4 devoluções para hoje** (Figura 12). Na tela de retirada, o botão **Confirmar retirada** fica desabilitado e a barra inferior lista o que falta ("Falta preencher: ferramenta, colaborador, setor de destino") (Figura 13). O código da ferramenta e a matrícula são identificados pela API (`/ferramentas/por-codigo` , `/colaboradores/identificar`): a tela mostra a ferramenta e o colaborador reais, com o setor já preenchido (Figura 14). Ao confirmar, a API responde **201**, aparece o toast "Devolver hoje · destino Controle de Qualidade" e a confirmação visual (Figura 15). Voltando ao dashboard, os números mudam sem ninguém recarregar a página: **58 disponíveis, 23 em uso, 5 para hoje** (Figura 16).

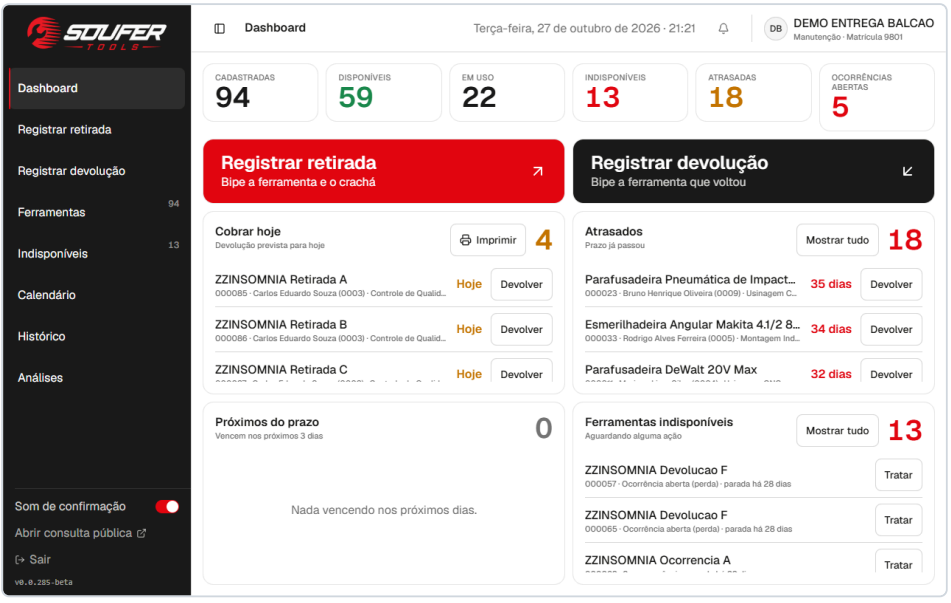


Figura 12 — Dashboard antes da retirada (59 disponíveis, 22 em uso)

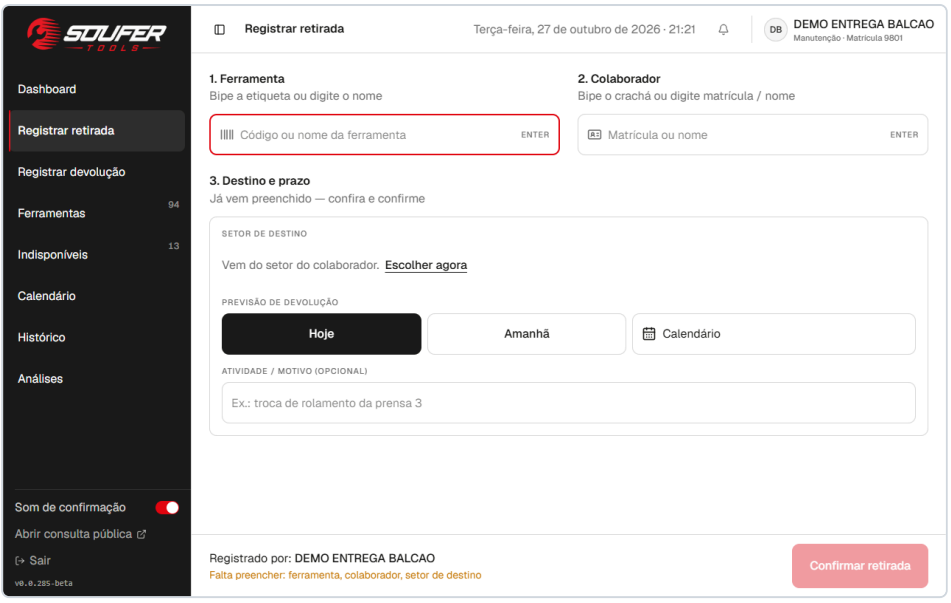


Figura 13 — Retirada: botão bloqueado e lista do que falta

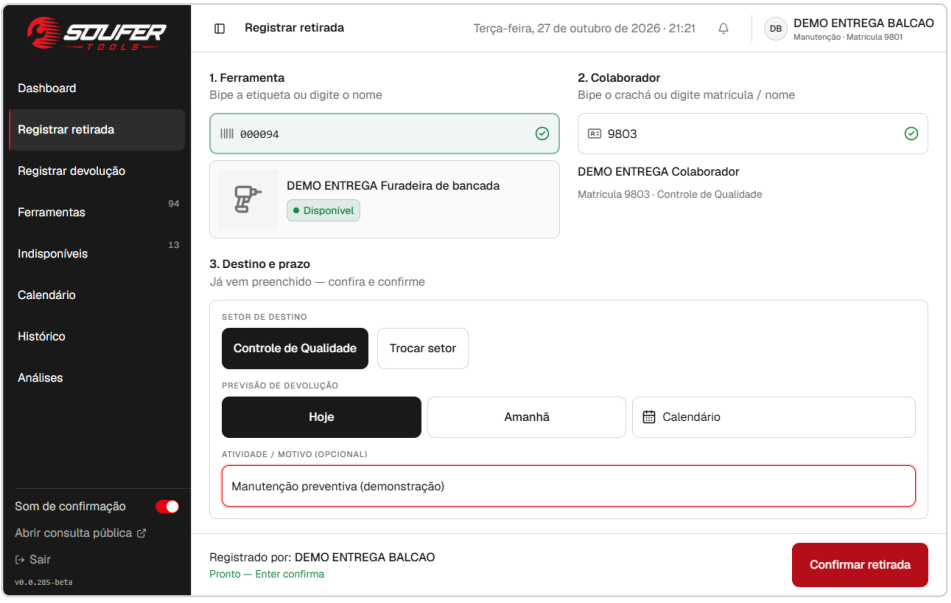


Figura 14 — Retirada: ferramenta e colaborador identificados pela API

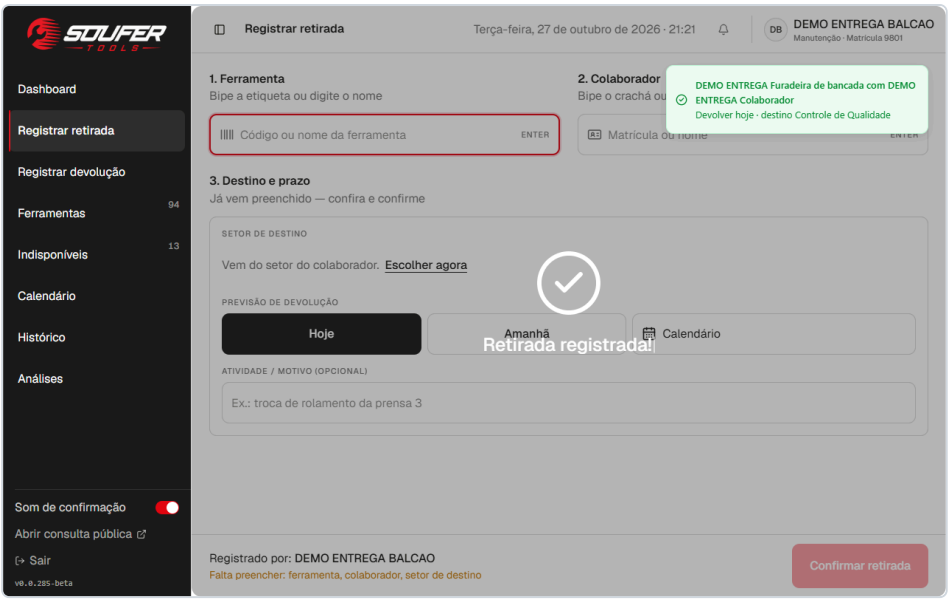


Figura 15 — Retirada registrada (201)

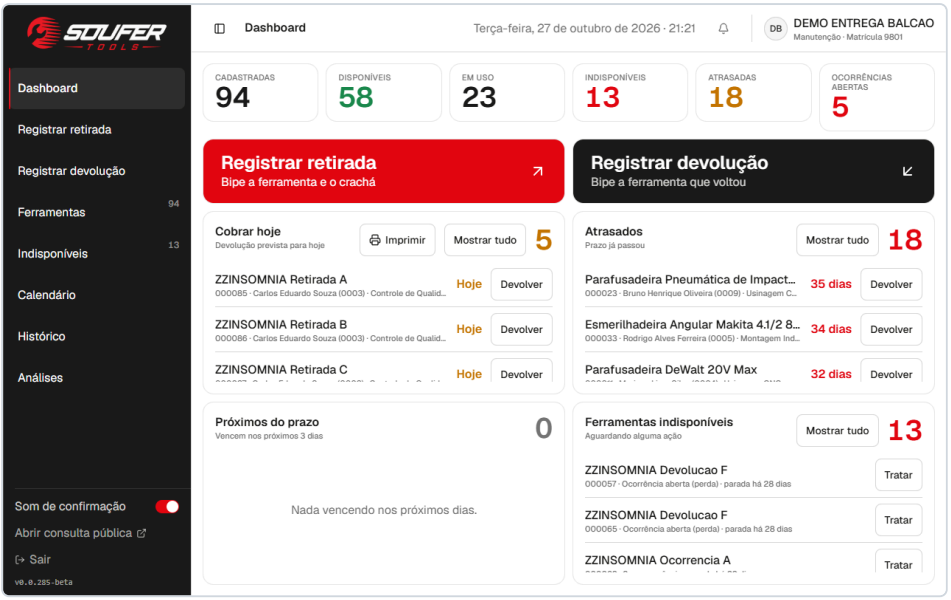


Figura 16 — Dashboard depois da retirada (58 disponíveis, 23 em uso)

Fluxo 5 — Devolução com avaria e abertura de ocorrência (PATCH /emprestimos/:id/devolucao)

Na devolução, a ferramenta é localizada pelo código e a tela mostra o empréstimo aberto (quem retirou, setor, atividade, saída e prazo) (Figura 17). Ao marcar **Avaria**, a descrição passa a ser obrigatória e o botão muda para "Confirmar e abrir ocorrência", bloqueado até o preenchimento (Figura 18). Com a descrição (Figura 19), a API responde **200**, registra a devolução, **abre a ocorrência automaticamente** em nome do colaborador e move a ferramenta para Indisponível; o toast confirma "Ocorrência de avaria aberta em nome de DEMO ENTREGA Colaborador" (Figura 20).

A interface reflete tudo sem recarregar: o contador de **Indisponíveis** no menu lateral sobe de 13 para 14, a ferramenta aparece na tela **Indisponíveis** com a ocorrência aberta e a linha do tempo da tratativa (Figura 21), o dashboard soma 14 indisponíveis e volta a 22 em uso (Figura 22), e o detalhe da ferramenta mostra o status Indisponível, o histórico do empréstimo (Devolvido · Avaria) e a ocorrência (Figura 23).

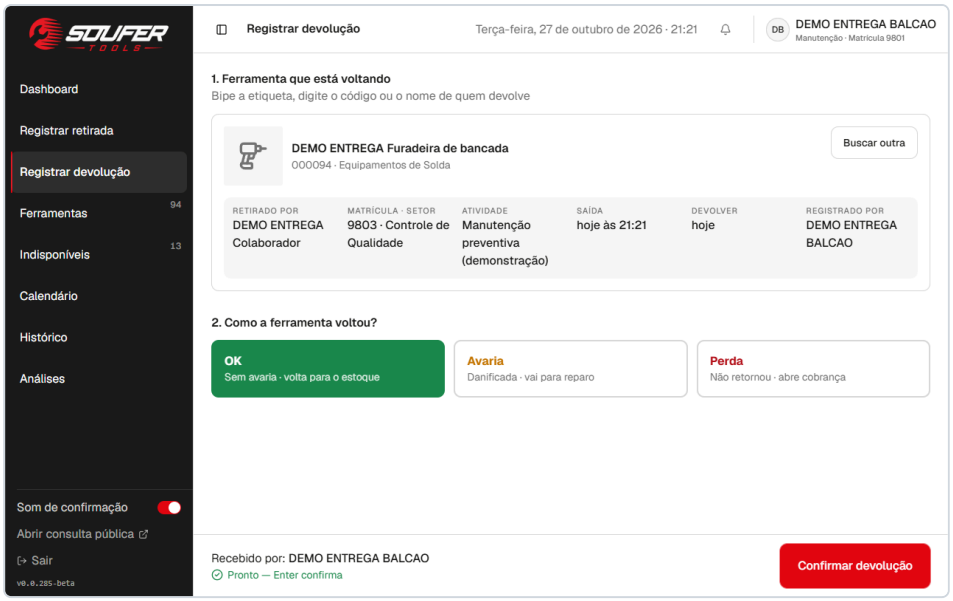


Figura 17 — Devolução: empréstimo aberto localizado pela API

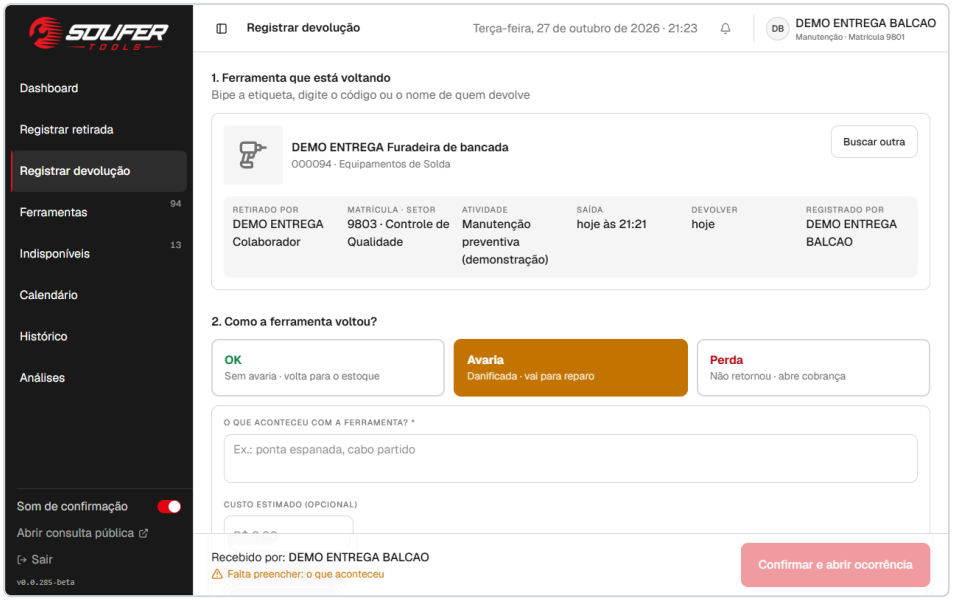


Figura 18 — Devolução: avaria exige descrição

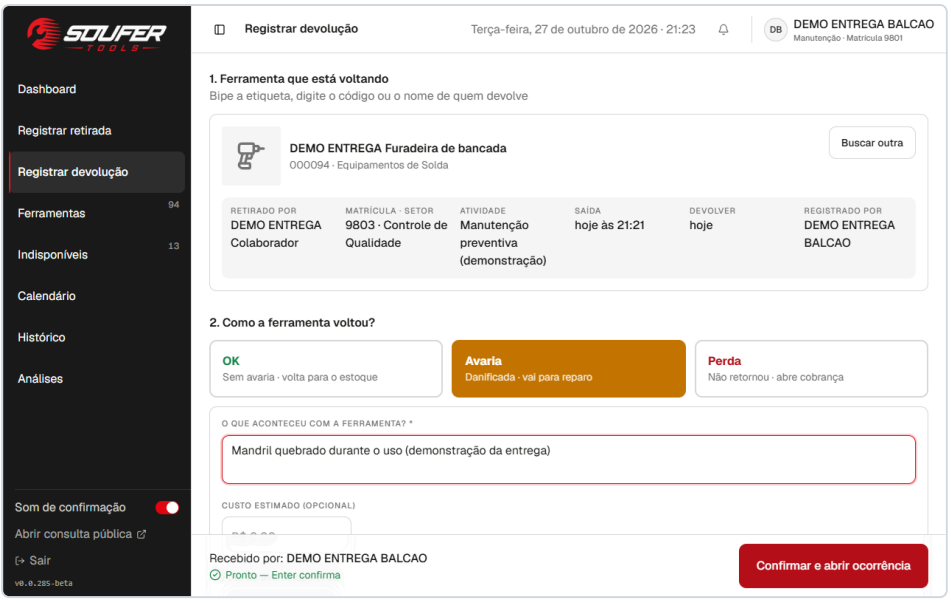


Figura 19 — Devolução: avaria descrita

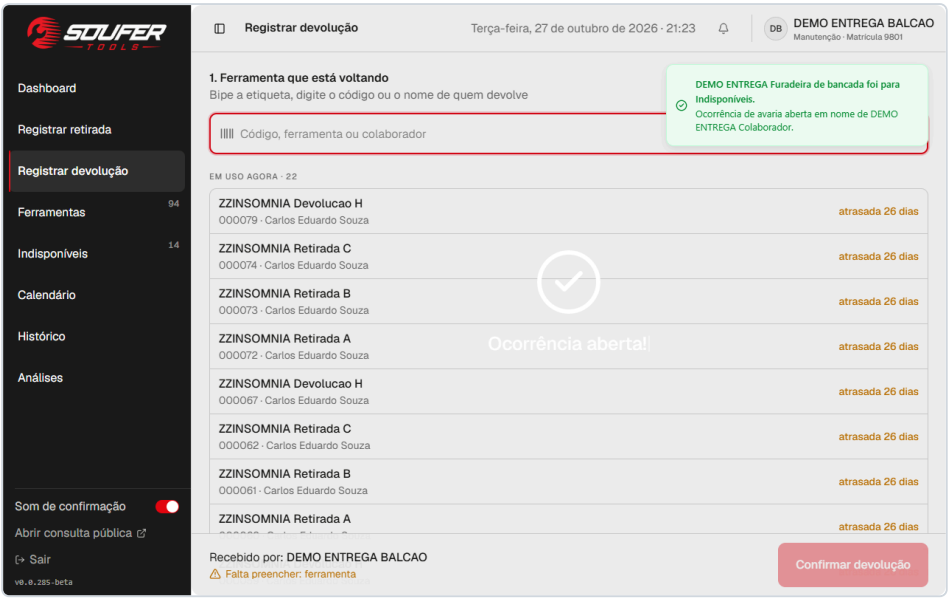


Figura 20 — Devolução registrada e ocorrência aberta

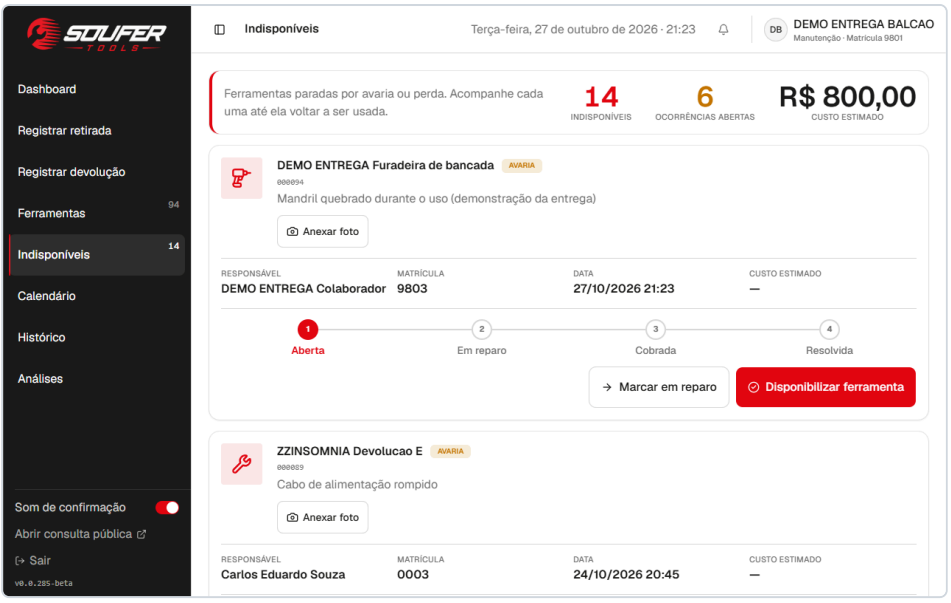


Figura 21 — Indisponíveis: ocorrência recém-aberta com a linha do tempo da tratativa

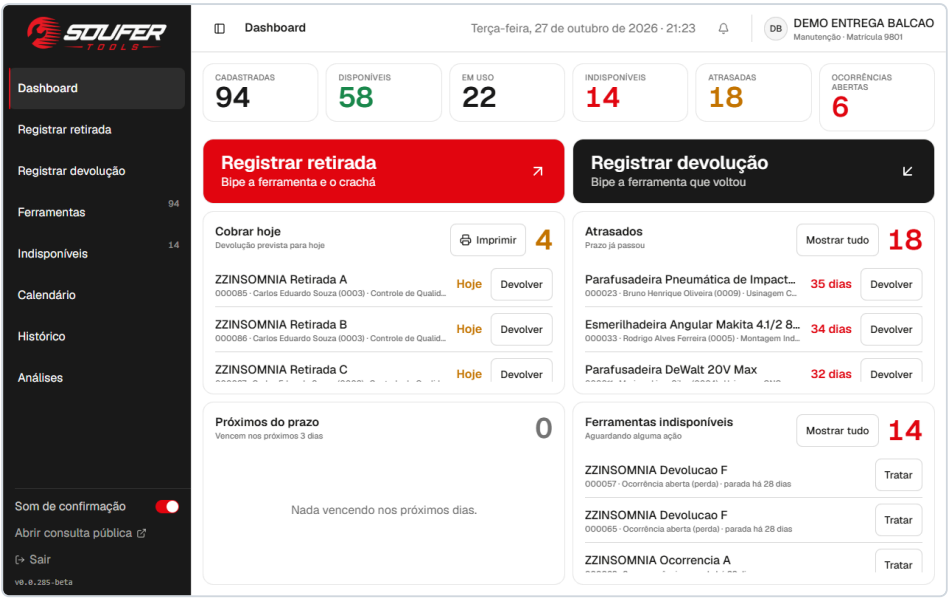


Figura 22 — Dashboard depois da devolução (14 indisponíveis)

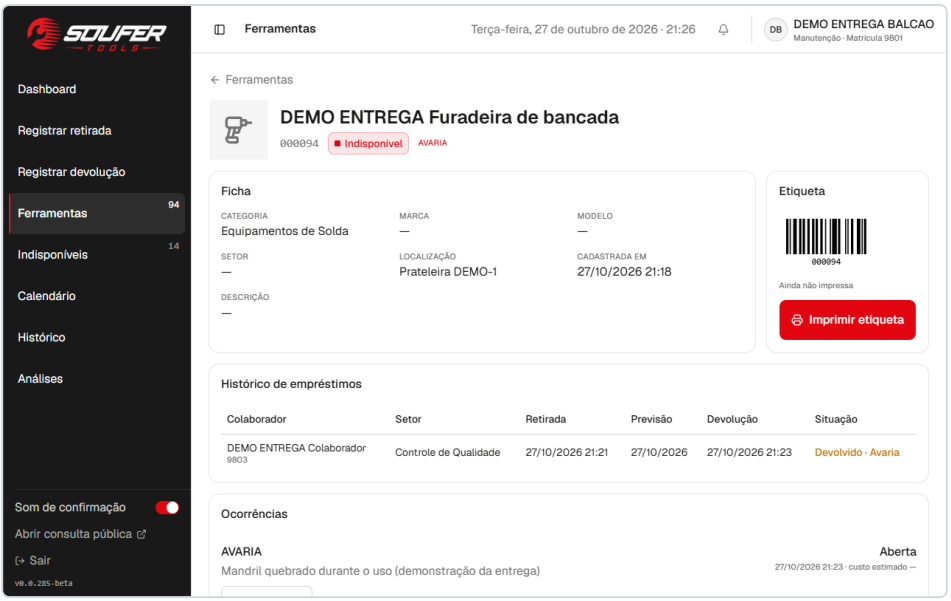


Figura 23 — Detalhe da ferramenta: status, histórico e ocorrência

O que é invalidado em cada operação

Operação	Cache invalidado (`invalidateQueries`)	Efeito visível
Cadastrar ferramenta	`ferramentas`	Lista, contador do menu e dashboard
Retirar ferramenta	`ferramentas` , `emprestimos` , `notificacoes`	Status "em uso", prazos, sino
Devolver ferramenta	`ferramentas` , `emprestimos` , `notificacoes`	Volta ao estoque ou vai para Indisponíveis
Cadastro rápido de colaborador	`colaboradores`	Colaborador já disponível na retirada
Avançar tratativa / disponibilizar / baixar	`ferramentas` e a chave da ocorrência	Linha do tempo e status
Enviar foto da ocorrência	`ferramentas` , `emprestimos`	Foto no histórico
Salvar ou inativar cadastro	chave do recurso	Lista de cadastros
Marcar notificação como lida	`notificacoes`	Contador do sino

7. Tratamento de erros e perfis

Situação	Resposta da API	O que o usuário vê
Dados inválidos	`400 VALIDATION_ERROR` com `details` por campo	Mensagem ao lado do campo; o formulário não fecha
Credencial errada	`401 INVALID_CREDENTIALS`	"Matrícula ou senha inválidas." e PIN limpo
Sessão expirada ou sem token	`401`	Interceptor encerra a sessão e leva ao login
Sem permissão para a ação	`403 ACCESS_DENIED`	Aviso vermelho; ações de admin nem aparecem no menu
Registro inexistente	`404`	Mensagem da API ou tela "Página não encontrada"
Duplicidade ou conflito de estado	`409 DUPLICATE_ENTRY`	Aviso vermelho, mantendo o que foi digitado
API fora do ar	sem resposta	Texto padrão de erro de conexão e tela de erro inesperado

Perfis (403). O sistema tem três perfis. A edição e a inativação de cadastros são exclusivas do `admin`. Na API, um usuário de manutenção que tenta editar um colaborador ou uma ferramenta recebe **403** `ACCESS_DENIED` (`evidencias/01-api-respostas-de-erro.txt`). No front, as telas de cadastro nem aparecem no menu de quem não é admin e, pela URL direta, respondem a tela de "não disponível" (Figura 24).

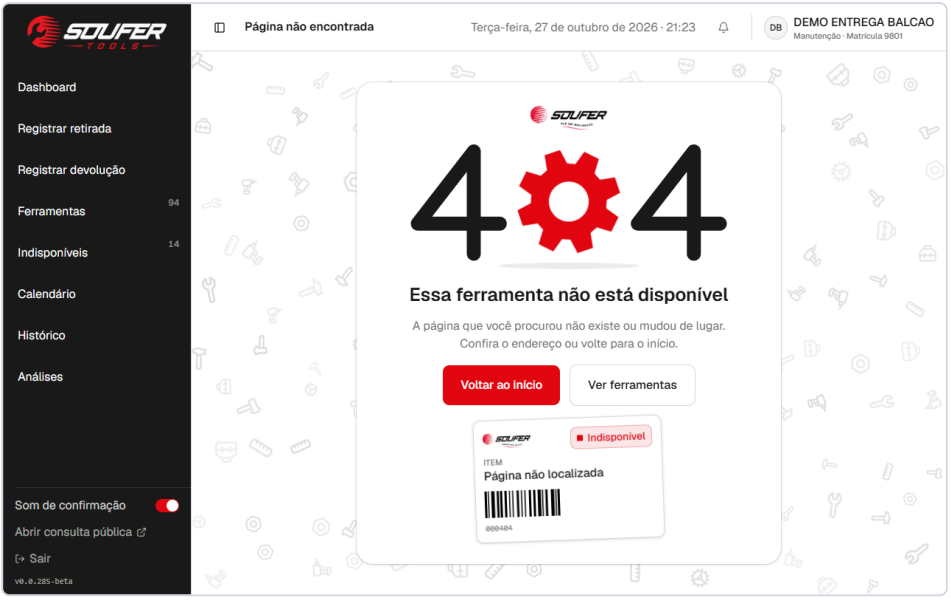


Figura 24 — Perfil manutenção acessando /cadastros/colaboradores pela URL: tela não disponível

8. Responsividade (360 px e 1280 px)

As telas foram verificadas em 360 px (celular) e em 1280 px (desktop; todos os prints das seções anteriores são em 1280 px). A página de retirada, por exemplo, não gera rolagem horizontal em 360 px. Em 360 px o

menu lateral vira o botão de menu do cabeçalho e os cartões e filtros se reorganizam.



Figura 25 — 360 px: login com validação



Figura 26 — 360 px: dashboard



Figura 27 — 360 px: retirada com colaborador identificado pela API



Figura 28 — 360 px: indisponíveis

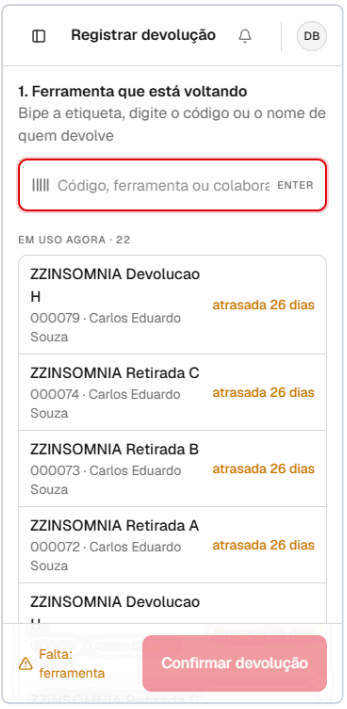


Figura 29 — 360 px: devolução

9. Testes automatizados

Suíte	Resultado	Arquivo
Front (`web`, Vitest)	11 arquivos, **72 testes passando** (cliente HTTP e interceptor de 401, hooks de retirada e ferramentas, importação de CSV, formatação, patrimônio, análises)	`evidencias/02-testes-web-vitest.txt`
API (`api`, Vitest + Supertest)	42 arquivos, **437 testes passando**	`evidencias/03-testes-api-vitest.txt`

O `npm install` e o `npm run build` rodam sem erro no `web` e na `api`.

10. Limitações conhecidas

- **Ambiente dos prints:** as evidências vêm do ambiente local (mesmo código da branch `development`, banco `soufer_dev`), e não do servidor publicado, para não gravar dados de teste no ambiente de produção.
- **Mensagem de 409 genérica:** o aviso de matrícula duplicada é o texto padrão "Registro duplicado ou operação em conflito com o estado atual", sem citar qual campo repetiu (o campo vem em `details` da API, mas a tela ainda não o usa).
- **Importação de CSV:** não tem tela própria. É um botão "Importar CSV" nas telas de cadastro que abre um diálogo de cadastro em lote.
- **CRUD de atividades:** o cadastro de atividades ainda não tem tela (a atividade é um texto opcional na retirada).
- **Status "Extraviada" e o termo "Extravio":** previstos para uma próxima etapa; hoje a interface usa "perda".

- **Tela de Indisponíveis:** ainda monta o histórico de cada ferramenta com uma chamada por item; a troca por uma única consulta de ocorrências está planejada.
- **360 px:** o valor do cartão "Custo estimado" em Indisponíveis fica justo na borda do cartão em telas muito estreitas.

11. Como reproduzir

```
# API
cd api && npm install && cp .env.example .env   # preencher banco e JWT_SECRET
npm run db:migrate && npm run dev                # http://localhost:3000/v1

# Front
cd web && npm install && cp .env.example .env    # VITE_API_URL=http://localhost:3000/v1
npm run dev                                       # http://localhost:5173

# Testes e build
npm test && npm run build
```